DAY 28 | HANDS-ON ACTIVITIES

Agent Protocols (MCP, A2A) and Advanced Agents: Problem Statements

Eight problems. The MCP and A2A parts run locally in Colab and VS Code; the data agent runs offline with templates or online with a model. Offline results show how the code works, not real model quality.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: MCP or A2A?

Level: Warm-up | **Concepts:** protocol choice

Decide which protocol fits: give an assistant access to a CRM; let a procurement agent ask a supplier's agent for a quote; expose a document store; delegate travel booking to a partner's agent.

Sample run / expected result

```
CRM access: MCP
Supplier quote: A2A
Document store: MCP (resources)
Travel booking: A2A
```

Hints

- Tools and data: MCP. Another agent with its own reasoning: A2A.

Problem 2: Your First MCP Server on mcp 2.x NEW (v4)

Level: Easy | **Concepts:** MCPServer, mcp.Client

Build an MCP server with MCPServer and tools get_ticket and list_open_tickets. Connect with mcp.Client in-process, list the tools, call both, and print the protocol version.

Sample run / expected result

```
protocol: 2026-07-28
tools: ['get_ticket', 'list_open_tickets']
get_ticket(T-1001) -> open, P2, network
```

Hints

- from mcp.server.mcpserver import MCPServer
- async with Client(mcp) as c: ...

Problem 3: Resources and Prompts

Level: Medium | **Concepts:** MCP resources, prompts

Add an SLA policy resource and a prompt template for apology emails. List and read them from the client.

Sample run / expected result

```
resources: ['policy://sla']
prompts: ['apology_email']
```

Hints

- @mcp.resource("policy://sla") and @mcp.prompt()

Problem 4: MCP Tools for an Agent NEW (v4)

Level: Medium | **Concepts:** tool wrapping, create_agent

Wrap the MCP tools as LangChain tools and give them to a create_agent assistant. Ask about a ticket and print which MCP tool was called. Then write the Codex config entry for the same server.

Sample run / expected result

```
agent called MCP tool: get_ticket {'ticket_id': 'T-1001'}
[mcp_servers.smartdesk]
command = "python"
args = ["mcp_server.py"]
```

Hints

- list_tools() gives name, description and input_schema.
- Read-only tools for Codex.

Problem 5: Task Lifecycle NEW (v4)

Level: Medium | **Concepts:** MCP Tasks

Simulate a slow export tool that returns a task. Poll it until completed, then fetch the result. Add a cancel path and a TTL.

Sample run / expected result

```
task 4f2a: working -> working -> completed
result: 120 tickets exported
cancelled task: cancelled
```

Hints

- Statuses: working, input_required, completed, failed, cancelled.

Problem 6: Signed Agent Card NEW (v4)

Level: Medium | **Concepts:** A2A v1.0, signatures

Sign the refund agent's card with an Ed25519 key. The support agent verifies the card before delegating. Change one skill in the card and show verification fails.

Sample run / expected result

```
card verified: True -> delegating task
tampered card verified: False -> refuse to delegate
```

Hints

- Sign the canonical JSON (sorted keys) of the card without its signatures field.

Problem 7: Data Analysis Agent in a Sandbox NEW (v4)

Level: Challenging | **Concepts:** AST checks, subprocess limits

Ask five questions of the ticket dataset. Run each piece of generated code in a separate process with a time limit and no access to your files. Show the code, the result and a one-line explanation. Try three malicious snippets and a never-ending loop.

Sample run / expected result

```
Q: city with most P1 tickets? -> Osaka
Blocked: import os
Blocked: open(...)
Killed: time limit 5s
```

Hints

- Block imports, dunder attributes, open, exec and eval.
- Run the Docker version on your laptop.

Problem 8: Handoff Payload

Level: Stretch | **Concepts:** Pydantic, handoff design

Define a Handoff model and create a handoff from the bot to a human for an angry customer whose credit exceeds the bot's limit. Send it as an A2A-style task artifact.

Sample run / expected result

```
Handoff(reason='credit above bot limit', urgency='high', steps_tried=[...], suggested_next_st
```

Hints

- Write it so the receiver needs no further context.